

UPI ATM - Frontend Integration Guide

Azoopay backend · UPI ATM sub-merchant onboarding and UPI ATM QR cash withdrawal

Version 1.0 · Prepared 3 October 2026 from the backend source (internal/handlers, internal/store, internal/models, internal/routes).

How to use this document

This guide is written so that it can be handed, together with the frontend codebase, to a developer or to an AI coding assistant (e.g. Claude) to build the UPI ATM screens. Every endpoint, request field, response field and status value below is taken directly from the backend code. Where the backend behaves in a way the UI must compensate for, it is called out in a **Frontend note**.

Instruction for the implementer (human or AI): follow the existing patterns of the frontend codebase - its HTTP client / axios instance, auth-token handling, form components, toast/error display, table and pagination components, and route guards. The UPI ATM module is a sibling of the existing **AEPS** module and deliberately mirrors its API shape, so the AEPS onboarding and AEPS transaction screens are the closest reference implementation in the frontend. Reuse them (especially the RD-service biometric capture code) rather than writing new infrastructure.

Feature summary

- **Onboarding (one time per retailer):** retailer applies -> admin approves -> merchant signup with Payntric (min-KYC) -> eKYC status check -> biometric KYC (only if required) -> poll eKYC status until APPROVED.
- **Transactions:** an onboarded retailer generates a dynamic UPI QR for an amount; the customer scans and pays from any UPI app; the frontend polls the status; on SUCCESS the backend credits the amount to the retailer's wallet and the retailer hands over cash.
- **Reports:** transaction history for a retailer and for admin, with date / status / search filters and pagination.

1. Conventions

1.1 Base URL and authentication

All paths in this document are relative to the backend base URL already used by the frontend (the Go server listens on SERVER_PORT). Every UPI ATM endpoint is protected and requires the JWT obtained at login:

```
Authorization: Bearer <jwt-token>
Content-Type: application/json
```

Missing / malformed / expired tokens return **401** with `{"error": "..."}.` Handle this exactly as the rest of the app does (usually: clear session and redirect to login).

1.2 The {id} path parameter

Wherever a path contains `{id}`, it is the **retailer ID** (e.g. R0001) - for both retailer and admin screens. On retailer screens use the logged-in retailer's ID; on admin screens use the ID of the row selected.

1.3 Response envelope

Successful responses are HTTP **200** and are a JSON object with a message string plus one data key whose name varies per endpoint (listed per endpoint below). Errors use a single `error` key:

HTTP status	Body	Meaning / what to show
200	<code>{"message": "...", "<dataKey>": ... }</code>	Success.

HTTP status	Body	Meaning / what to show
400	{ "error": "<reason>" }	Validation failure or a Payntric (provider) error. The reason is human readable (e.g. <i>"Amount must be between 50.0 and 50000.0"</i> , <i>"The given Name is not matching with pan records..."</i>). Show it to the user as-is.
401	{ "error": "<reason>" }	Auth problem. Redirect to login.
500	{ "error": "internal server error" }	Database/server failure. Also returned when a record does not exist (e.g. retailer has no UPI ATM application or merchant record yet). See the Frontend note below.

Frontend note: the backend does not return 404. For the "get application" and "get merchant" lookups, treat a 500 as *"not found / not started yet"* and show the next onboarding step instead of an error banner. For all other endpoints show a generic "Something went wrong, please try again".

Frontend note: list endpoints return null (not []) when there are no rows. Always normalise with data ?? [].

1.4 Location

Latitude and longitude are sent as **strings** (e.g. "28.620370"). Obtain them from `navigator.geolocation.getCurrentPosition` and convert with `String(coords.latitude)`. Ask for permission before the step that needs them and block submission with a clear message if permission is denied.

2. Onboarding

2.1 Flow and screen states

Build a single **UPI ATM Onboarding** page for the retailer that decides what to show from two lookups: the application (2.3.3) and the merchant record (2.3.7). Suggested state machine:

#	Condition	Retailer UI	Action / API
1	Application lookup fails (no application)	"Apply for UPI ATM" form (remarks + auto location)	POST /upi-atm/onboarding/apply
2	Application status = PENDING	"Application under review"	Nothing; admin acts
3	Application status = REJECTED	Show admin_remarks; allow re-contact/support	-
4	Application APPROVED, merchant lookup fails	"Complete signup" button (shows the retailer's KYC details read-only for confirmation)	GET /upi-atm/onboarding/signup/{id}
5	Merchant ekyc_status = PENDING	"Check eKYC status" button	GET /upi-atm/onboarding/check/ekyc/{id}
6	ekyc_action = ACTION-REQUIRED and ekyc_status = PENDING	Biometric capture screen (fingerprint via RD service)	POST /upi-atm/onboarding/bio/kyc/{id}
7	ekyc_status = APPROVAL_PENDING	"Waiting for bank approval" + Refresh button	GET /upi-atm/onboarding/check/ekyc/{id}
8	ekyc_status = APPROVED and not blocked	"UPI ATM active" - link to the QR screen	-
9	is_merchant_blocked = true	"UPI ATM disabled, contact support"	-

Frontend note: the backend does **not** enforce that the application is approved before signup. The UI must only show the signup action when `upi_atm_application_status` is the approved value. Use the same status strings as the existing AEPS onboarding UI (the database default for a new application is PENDING; the admin screen sets e.g. APPROVED / REJECTED - whatever value the admin sends is stored verbatim).

Frontend note: signup uses the KYC data already stored on the retailer profile (name, phone, email, PAN, Aadhaar, address, city, pincode, date of birth, gender) and the location saved on the application. If Payntrix rejects it (e.g. name/PAN mismatch, invalid mobile, business PAN), the error comes back as a 400 with the provider message; tell the retailer to correct their profile KYC.

2.2 eKYC status values

Field	Value	Meaning
ekyc_status	PENDING	eKYC not done yet.
ekyc_status	APPROVAL_PENDING	Biometric submitted; waiting for bank approval. Keep polling eKYC status.
ekyc_status	APPROVED	Fully onboarded - QR generation allowed.
ekyc_action	ACTION-REQUIRED	Biometric authentication must be captured and submitted.
ekyc_action	NO-ACTION-REQUIRED	Nothing more to do.
min_kyc_status	COMPLETED	Signup (min-KYC) done.

2.3 Endpoints

2.3.1 Apply

POST /upi-atm/onboarding/apply

Used by: Retailer. Creates the UPI ATM application (one per retailer).

```
{
  "retailer_id": "R0001",
  "latitude": "28.620370",
  "longitude": "77.381041",
  "retailer_remarks": "Please enable UPI ATM"
}
```

Response: { "message": "upi atm application created successfully" }. A second apply for the same retailer fails (500).

2.3.2 Change application status

POST /upi-atm/onboarding/app/change/status/{id}

Used by: Admin. Approve / reject an application. Empty strings leave a field unchanged.

```
{
  "retailer_id": "R0001",
  "upi_atm_application_status": "APPROVED",
  "admin_remarks": "Verified",
  "retailer_remarks": ""
}
```

Frontend note: the retailer is identified by retailer_id in the **body**; put the same ID in the path. Response: { "message": "upi atm application status updated successfully" }.

2.3.3 Get application (single / all)

GET /upi-atm/onboarding/get/application/{id}

Used by: Retailer, Admin. Data key: application (object).

GET /upi-atm/onboarding/get/applications

Used by: Admin. Data key: applications (array or null), newest first.

```
{
  "message": "upi atm application fetched successfully",
  "application": {
    "upi_atm_application_id": "1",
    "retailer_id": "R0001",
    "upi_atm_application_status": "PENDING",
    "retailer_remarks": "Please enable UPI ATM",
    "admin_remarks": "",
    "latitude": "28.620370",
    "longitude": "77.381041",
    "retailer_details": {
      "retailer_name": "Rakesh Kumar",
      "retailer_phone": "9876543210",
      "retailer_email": "rakesh@example.com",
      "retailer_aadhaar_number": "XXXXXXXX1234",
      "retailer_pan_number": "ABCDE1234F",
      "retailer_full_address": "A 64 Sector 63",
      "retailer_city": "Noida",
      "retailer_pincode": "201306",
      "retailer_date_of_birth": "1990-05-14T00:00:00Z",
      "retailer_gender": "MALE"
    },
    "created_at": "2026-10-03T10:15:00Z",
    "updated_at": "2026-10-03T10:15:00Z"
  }
}
```

Frontend note: Aadhaar/PAN are returned in full - mask them in the UI (show last 4).

2.3.4 Merchant signup (min-KYC)

GET /upi-atm/onboarding/signup/{id}

Used by: Retailer (after approval). No body. Registers the retailer with Payntric and saves the sub-merchant / outlet IDs. Data key: `api_response`.

```
{
  "message": "upi atm merchant signup successfull",
  "api_response": {
    "status": "SUCCESS",
    "statusCode": "TXN",
    "message": "Min-KYC completed",
    "subMerchantId": "SUBM_3BF7C7BD",
    "parentMerchantId": "MCH42CB2E",
    "outletId": "612346",
    "minKycStatus": "COMPLETED",
    "ekycStatus": "PENDING",
    "mobileChangeState": "NONE",
    "ipayUuid": "h000af22-....",
    "timestamp": "2026-06-19T11:54:20.278661103",
    "data": { "name": "...", "dateOfBirth": "...", "gender": "M", "pincode": "...",
      "state": "UTTAR PRADESH", "city": "...", "address": "..." }
  }
}
```

Typical 400 errors: *Only individual PAN is allowed; The given Name is not matching with pan records...; Validation failed* (invalid mobile). Calling signup twice fails (400 from the provider, or 500 if the merchant record already exists). After success, immediately call the eKYC status check (2.3.5).

2.3.5 Check eKYC status

GET `/upi-atm/onboarding/check/ekyc/{id}[?gw=AC|NA|JA]`

Used by: Retailer. Asks Payntric whether biometric KYC is needed / approved and saves the result (including the referenceKey needed for biometric KYC). Data key: `api_response`.

Optional query gw selects the bank gateway: AC = Airtel Payments Bank (default), NA = NSDL, JA = Jio Payments Bank. Omit it unless the business tells you otherwise; if exposed, use the same value on every call for that retailer.

```
// Biometric required
{
  "message": "ekyc check successfull",
  "api_response": {
    "status": "SUCCESS", "statusCode": "TXN",
    "message": "Biometric authentication required",
    "subMerchantId": "SUBM_B5...", "outletId": "612346",
    "minKycStatus": "COMPLETED",
    "ekycStatus": "PENDING",
    "ekycAction": "ACTION-REQUIRED",
    "referenceKey": "xxxxxxxx",
    "data": { "status": "PENDING", "isFaceAuthAvailable": true,
      "isBiometricKycMandatory": false, "bankName": "Airtel Payments Bank" }
  }
}

// Already approved
{ "message": "ekyc check successfull",
  "api_response": { "status": "SUCCESS", "message": "Biometric authentication already completed",
    "ekycStatus": "APPROVED", "ekycAction": "NO-ACTION-REQUIRED",
    "data": { "status": "APPROVED", "bankName": "Jio Payments Bank" } } }
```

Drive the UI from `api_response.ekycStatus` and `api_response.ekycAction` (state table in 2.1). In the APPROVAL_PENDING state, offer a manual Refresh and/or poll every 30-60 s while the page is open - bank approval can take minutes to hours, do not poll aggressively.

2.3.6 Submit biometric KYC

POST `/upi-atm/onboarding/bio/kyc/{id}`

Used by: Retailer. Only when ekycAction = ACTION-REQUIRED. Requires a prior eKYC check (otherwise 400 reference key not found, check ekyc status first). Data key: api_response.

```
{
  "latitude": "28.620370",
  "longitude": "77.381041",
  "capture_type": "FMR", // optional, defaults to "FMR"
  "biometric_data": {
    "encryptedAadhaar": "",
    "dc": "", "dpId": "", "mc": "", "mi": "", "rdsId": "", "rdsVer": "",
    "ci": "", "sessionKey": "", "pidData": "", "pidDataType": "", "hmac": "",
    "errCode": "0", "errInfo": "", "fCount": "1", "fType": "2",
    "iCount": "0", "iType": "", "pCount": "0", "pType": "",
    "qScore": "", "nmPoints": "", "srno": "", "sysid": "", "ts": "2026-10-03T10:30:00"
  }
}
```

All biometric_data values are **strings**. The keys are identical to the existing AEPS biometric_data object, so reuse the AEPS fingerprint-capture helper (RD service call + PID XML parsing). Mapping from the RD service PID XML:

biometric_data key	PID XML source
errCode, errInfo, fCount, fType, iCount, iType, pCount, pType, nmPoints, qScore	<Resp> attributes of the same names
dpId, rdsId, rdsVer, mi, mc, dc	<DeviceInfo> attributes
srno, sysid	<DeviceInfo><additional_info><Param name="srno" / "sysid"> value
ci	<Skey ci="..."> attribute
sessionKey	<Skey> text content
hmac	<Hmac> text content
pidDataType	<Data type="..."> attribute (e.g. X / P)
pidData	<Data> text content
ts	Capture timestamp, format YYYY-MM-DDTHH:mm:ss
encryptedAadhaar	Empty string unless the business provides one

If the RD service returns errCode other than "0", show errInfo and let the user recapture - do not submit. On success the response has ekycStatus: "APPROVAL_PENDING"; move to the waiting state and poll eKYC status. A 400 such as *The selected captureType is invalid.* is a provider error - show it.

2.3.7 Merchant details (single / all) and blocking

GET /upi-atm/onboarding/get/merchant/{id}

Used by: Retailer, Admin. Data key: merchant (object).

GET /upi-atm/onboarding/get/merchants

Used by: Admin. Data key: merchants (array or null), newest first.

```
{
  "message": "upi atm merchant details fetched successfully",
  "merchant": {
    "upi_atm_merchant_id": "1",
    "retailer_id": "R0001",
    "sub_merchant_id": "SUBM_3BF7C7BD",
    "parent_merchant_id": "MCH42CB2E",
    "outlet_id": "612346",
    "min_kyc_status": "COMPLETED",
    "ekyc_status": "APPROVED",
    "ekyc_action": "NO-ACTION-REQUIRED",
    "reference_key": "xxxxxxxx",
    "status": "APPROVED",
    "is_face_auth_available": true,
  }
}
```

```
    "is_biometric_kyc_mandatory": false,  
    "bank_name": "Airtel Payments Bank",  
    "mobile_change_state": "NONE",  
    "ipay_uuid": "h000af22-...",  
    "is_merchant_blocked": false,  
    "created_at": "2026-10-03T10:20:00Z",  
    "updated_at": "2026-10-03T10:40:00Z"  
  }  
}
```

PATCH /upi-atm/onboarding/merchant/block/{id}

Used by: Admin. Block or unblock a merchant. Blocked merchants cannot generate QR codes.

```
{ "is_merchant_blocked": true }
```

Response: { "message": "upi atm merchant block status updated successfully" }.

3. UPI ATM transactions

3.1 Cash withdrawal flow (retailer)

- **Gate:** show the UPI ATM screen only when the merchant record has `ekyc_status = APPROVED` and `is_merchant_blocked = false`; otherwise link to onboarding. (The backend re-checks and returns 400.)
- **Form:** amount (Rs. 50 - 50,000) and customer mobile (10 digits). Validate on the client with the same rules.
- **Generate:** call Create QR (3.2). Render `response.qrString` as a QR code image (e.g. a QR library such as `qrcode.react / qrcode`, whichever the project already uses). On mobile, also offer `response.qrMobile` as a "Pay with UPI app" deep link.
- **Countdown:** show a timer using `displayExpirySec` (validity in seconds) / `expiryDt`. Keep `response.requestId` in state - it is the only key needed for status checks.
- **Poll:** call QR Status (3.3) every 3-5 seconds while `qrStatus` is INITIATED or PENDING. Stop on SUCCESS or FAILED. When the timer expires, do one or two final checks (payments can land at the last second) and then show "QR expired" with a "Check again" button.
- **Result:** on SUCCESS show a success receipt (amount, requestId, transactionId, time) and the message *"Amount credited to your wallet - hand over cash to the customer"*; refresh the wallet balance shown in the header. On FAILED show failure and tell the retailer **not** to hand over cash.
- Stop polling when the component unmounts / the user navigates away. Disable the Generate button while a request is in flight to avoid duplicate QRs.

Wallet credit is done by the backend inside the status-check call, exactly once, on the first transition to SUCCESS. The frontend never calls a separate credit API. Re-checking an already successful transaction is safe and will not credit twice.

3.2 Create QR

POST `/upi-atm/create-qr/{id}`

Used by: Retailer. Data key: `response`; message is the provider message.

```
{
  "amount": 100,
  "mobile": "6396978580",
  "latitude": "28.620382",      // optional - device location
  "longitude": "77.381043"     // optional - send both or neither
}
```

If latitude/longitude are omitted the backend uses the location saved on the application. Prefer sending the current device location.

```
{
  "message": "QR Generated Successfully.",
  "response": {
    "status": "SUCCESS",
    "statusCode": "TXN",
    "message": "QR Generated Successfully.",
    "transactionId": "UATM_586D556DBC604505",
    "requestId": "4f6c2b8e-1d2a-4c55-9a7e-0b1f2c3d4e5f",
    "qrStatus": "INITIATED",
    "ipayId": "P260619165646WCYHX",
    "qrString": "upi://pay?...",
    "qrMobile": "upi://pay?...",
    "expiryDt": "2026-06-19T16:57:30",
    "qrCreatedDt": "2026-06-19T16:56:47",
    "displayExpirySec": 30,
    "amount": 100.0,
    "payableValue": 100.0,
    "transactionValue": 100.0,
  }
}
```

```

    "commissionAmount": 0.0,
    "tdsAmount": 0.0,
    "netAmount": 0.0,
    "settlementStatus": "PENDING"
  }
}

```

400 error message	Cause
amount must be between 50 and 50000	Client-side validation missed (backend check).
mobile must be a 10-digit number	Client-side validation missed (backend check).
upi atm merchant is blocked	Admin blocked the merchant.
upi atm merchant ekyc is not approved	Onboarding not finished.
UPI-ATM not enabled for this merchant: ...	Provider has not enabled UPI ATM for this outlet - contact support.
Amount must be between 50.0 and 50000.0 / Amount must be greater than 100	Provider-side limits (may differ per merchant).
Invalid username or token. / System Error	Provider configuration/outage - show and allow retry later.

Frontend note: expiryDt and qrCreatedDt have no timezone (provider local time, IST). For the countdown, prefer displayExpirySec counted from when the response was received.

3.3 Check QR status

POST /upi-atm/qr-status

Used by: Retailer. Data key: response. Also persists the status and credits the wallet on SUCCESS.

```

{ "requestId": "4f6c2b8e-1d2a-4c55-9a7e-0b1f2c3d4e5f" }

{
  "message": "Transaction is pending",
  "response": {
    "status": "SUCCESS",
    "statusCode": "TXN",
    "message": "Transaction is pending",
    "transactionId": "UATM_CB4EDA80DE8A41A8",
    "requestId": "4f6c2b8e-1d2a-4c55-9a7e-0b1f2c3d4e5f",
    "qrStatus": "PENDING",
    "ipayId": "P260619175259TJQAH",
    "amount": 100.0,
    "settlementStatus": "PENDING"
  }
}

```

Important: decide the outcome from response.qrStatus, **not** response.status. status is "SUCCESS" even while the payment is pending, and is "FAILED" for a failed payment.

qrStatus	Meaning	UI action
INITIATED	QR generated	Keep polling
PENDING	Customer has not paid yet	Keep polling
SUCCESS	Payment received; wallet credited	Stop polling, show success receipt, refresh wallet balance
FAILED	Payment failed or QR expired	Stop polling, show failure, do not give cash

settlementStatus	Meaning
PENDING	Provider settlement pending (informational)
SETTLED / SUCCESS	Provider settlement complete
FAILED	Settlement failed

A 400 here (e.g. *Transaction not found.*, *System Error*) is a temporary/provider problem - keep polling a few more times before showing an error; a 500 means the requestId is unknown to the backend.

3.4 Transaction history

GET /upi-atm/get/{id}

Used by: Retailer. History for one retailer. Data key: response (array or null).

GET /upi-atm/get

Used by: Admin. History for all retailers. Data key: response (array or null).

Query parameters (all optional, same as the AEPS transaction reports):

Param	Format	Notes
limit	integer	Default 10.
offset	integer	Default 0. Page n => offset = (n-1) * limit.
start_date	YYYY-MM-DD	Interpreted as IST start of day.
end_date	YYYY-MM-DD	Inclusive (IST end of day).
status	string	Exact qrStatus filter: INITIATED, PENDING, SUCCESS, FAILED.
search	string	Partial match on requestId, transactionId, ipayId (admin list also matches retailer_id).

```
{
  "message": "upi atm transactions fetched successfully",
  "response": [
    {
      "retailer_id": "R0001",
      "retailer_name": "Rakesh Kumar",
      "transactionId": "UATM_CB4EDA80DE8A41A8",
      "requestId": "4f6c2b8e-1d2a-4c55-9a7e-0b1f2c3d4e5f",
      "qrStatus": "SUCCESS",
      "ipayId": "P260619175259TJQAH",
      "amount": 100,
      "payableValue": 100,
      "transactionValue": 100,
      "commissionAmount": 0,
      "tdsAmount": 0,
      "netAmount": 0,
      "settlementStatus": "SETTLED",
      "before_balance": { "Float64": 1500, "Valid": true },
      "after_balance": { "Float64": 1600, "Valid": true },
      "reason": { "String": "UPI-ATM", "Valid": true },
      "remarks": { "String": "UPI ATM Amount Credit To: R0001", "Valid": true },
      "created_at": "2026-10-03T10:45:00Z",
      "updated_at": "2026-10-03T10:46:10Z"
    }
  ]
}
```

Frontend note: before_balance, after_balance, reason and remarks are nullable objects (same shape as the AEPS transaction list): read `x.Valid ? x.Float64 : "-"` (or `.String`). They are only filled for SUCCESS transactions (the wallet credit). Note the mixed key casing (camelCase for provider fields, snake_case for ours) - use the keys exactly as shown. Timestamps are UTC ISO-8601; display in IST.

Suggested columns: Date/time, Request ID, Transaction ID, Amount, QR status (badge), Settlement status, Wallet before/after; admin view adds Retailer ID and name.

4. TypeScript types (reference)

Drop-in type definitions matching the JSON above. Adapt naming/location to the project's conventions.

```
type NullFloat = { Float64: number; Valid: boolean };
type NullString = { String: string; Valid: boolean };

export interface UpiAtmApplication {
  upi_atm_application_id: string;
  retailer_id: string;
  upi_atm_application_status: string; // PENDING | APPROVED | REJECTED ...
  retailer_remarks: string;
  admin_remarks: string;
  latitude: string;
  longitude: string;
  retailer_details: {
    retailer_name: string; retailer_phone: string; retailer_email: string;
    retailer_aadhaar_number: string; retailer_pan_number: string;
    retailer_full_address: string; retailer_city: string; retailer_pincode: string;
    retailer_date_of_birth: string; retailer_gender: string;
  };
  created_at: string; updated_at: string;
}

export type EkycStatus = "PENDING" | "APPROVAL_PENDING" | "APPROVED";
export type EkycAction = "ACTION-REQUIRED" | "NO-ACTION-REQUIRED";

export interface UpiAtmMerchant {
  upi_atm_merchant_id: string; retailer_id: string;
  sub_merchant_id: string; parent_merchant_id: string; outlet_id: string;
  min_kyc_status: string; ekyc_status: EkycStatus | ""; ekyc_action: EkycAction | "";
  reference_key: string; status: string;
  is_face_auth_available: boolean; is_biometric_kyc_mandatory: boolean;
  bank_name: string; mobile_change_state: string; ipay_uuid: string;
  is_merchant_blocked: boolean; created_at: string; updated_at: string;
}

export interface EkycApiResponse {
  status: string; statusCode: string; message: string;
  subMerchantId?: string; outletId?: string; minKycStatus?: string;
  ekycStatus: EkycStatus; ekycAction: EkycAction; referenceKey?: string;
  data: { status: string; isFaceAuthAvailable?: boolean;
    isBiometricKycMandatory?: boolean; bankName: string };
}

export interface BiometricData {
  encryptedAadhaar: string; dc: string; dpId: string; mc: string; mi: string;
  rdsId: string; rdsVer: string; ci: string; sessionKey: string;
  pidData: string; pidDataType: string; hmac: string;
  errCode: string; errInfo: string; fCount: string; fType: string;
  iCount: string; iType: string; pCount: string; pType: string;
  qScore: string; nmPoints: string; srno: string; sysid: string; ts: string;
}

export type QrStatus = "INITIATED" | "PENDING" | "SUCCESS" | "FAILED";

export interface CreateQrResponse {
  status: string; statusCode: string; message: string;
  transactionId: string; requestId: string; qrStatus: QrStatus; ipayId: string;
  qrString: string; qrMobile: string; expiryDt: string; qrCreatedDt: string;
  displayExpirySec: number; amount: number; payableValue: number;
  transactionValue: number; commissionAmount: number; tdsAmount: number;
  netAmount: number; settlementStatus: string;
}

export interface QrStatusResponse {
  status: string; statusCode: string; message: string;
  transactionId: string; requestId: string; qrStatus: QrStatus;
  ipayId: string; amount: number; settlementStatus: string;
}
```

```
export interface UpiAtmTransaction {
  retailer_id: string; retailer_name: string;
  transactionId: string; requestId: string; qrStatus: QrStatus; ipayId: string;
  amount: number; payableValue: number; transactionValue: number;
  commissionAmount: number; tdsAmount: number; netAmount: number;
  settlementStatus: string;
  before_balance: NullFloat; after_balance: NullFloat;
  reason: NullString; remarks: NullString;
  created_at: string; updated_at: string;
}
```

4.1 Polling helper (sketch)

```
// Poll every 4s until a terminal status, an expiry grace window, or unmount.
async function pollQr(requestId: string, expirySec: number,
  onUpdate: (r: QrStatusResponse) => void, signal: AbortSignal) {
  const deadline = Date.now() + (expirySec + 20) * 1000; // 20s grace after expiry
  while (!signal.aborted && Date.now() < deadline) {
    try {
      const { data } = await api.post("/upi-atm/qr-status", { requestId }, { signal });
      onUpdate(data.response);
      if (data.response.qrStatus === "SUCCESS" || data.response.qrStatus === "FAILED") return;
    } catch (e) { /* 400 = transient provider error: keep polling */ }
    await new Promise(r => setTimeout(r, 4000));
  }
}
```

5. Screens and implementation checklist

Retailer

- Menu entry "UPI ATM" next to AEPS. It routes to Onboarding until the merchant is APPROVED, then to the QR screen.
- UPI ATM Onboarding page implementing the state table in 2.1 (apply form, under-review, signup, eKYC check, biometric capture, waiting, active, blocked).
- UPI ATM Withdraw page: amount + mobile form, QR display with countdown, live status, success/failure receipt, wallet refresh.
- UPI ATM Transactions page: table with filters (date range, status, search) and pagination via limit/offset.

Admin

- UPI ATM Applications list (GET /upi-atm/onboarding/get/applications) with Approve / Reject actions (2.3.2) and remarks input.
- UPI ATM Merchants list (GET /upi-atm/onboarding/get/merchants) showing eKYC status, bank, outlet ID and a Block/Unblock toggle (2.3.7).
- UPI ATM Transactions report (GET /upi-atm/get) with filters and pagination.

Cross-cutting

- Use the existing API client so the Bearer token and base URL are applied automatically.
- Show error from 400 responses verbatim; generic message for 500 (except the not-found lookups in 1.3).
- Normalise null lists to [] and mask Aadhaar/PAN.
- Request location permission before Apply, Biometric KYC and Create QR.
- Never decide payment outcome from status; always use qrStatus.

6. Endpoint index

Method	Path	Who	Data key
POST	/upi-atm/onboarding/apply	Retailer	-
POST	/upi-atm/onboarding/app/change/status/{id}	Admin	-
GET	/upi-atm/onboarding/signup/{id}	Retailer	api_response
GET	/upi-atm/onboarding/check/ekyc/{id}	Retailer	api_response
POST	/upi-atm/onboarding/bio/kyc/{id}	Retailer	api_response
GET	/upi-atm/onboarding/get/applications	Admin	applications
GET	/upi-atm/onboarding/get/application/{id}	Retailer, Admin	application
GET	/upi-atm/onboarding/get/merchants	Admin	merchants
GET	/upi-atm/onboarding/get/merchant/{id}	Retailer, Admin	merchant
PATCH	/upi-atm/onboarding/merchant/block/{id}	Admin	-
POST	/upi-atm/create-qr/{id}	Retailer	response
POST	/upi-atm/qr-status	Retailer	response
GET	/upi-atm/get/{id}	Retailer	response
GET	/upi-atm/get	Admin	response